# APPLICATION DEVELOPMENT USING PYTHON PROGRAMMING LABORATORY

**Course Code: MCSL18** **Credits: 0:0:1**

1. **Python Basics, Control Structures and Functions, Strings, lists, list comprehensions:** Manipulate synthetic data records to categorize and process using control structures and string/list operations.

Python is a high-level, interpreted, general-purpose programming language. It is widely used in data science, machine learning, artificial intelligence, networking and automation.

| Concept | Description | Example |
|---|---|---|
| Variables | Store data values | name = "Ravi" |
| Integer | Whole numbers | age = 21 |
| Float | Decimal values | marks = 85.5 |
| String | Sequence of characters | "Python" |
| Boolean | True or False | status = True |
| Input | Accept user data | input() |
| Output | Display results | print() |
| Type conversion | Convert one type to another | int("25") |
| Operators | Perform operations | +, -, *, /, % |

```
# Student information

name = "Rahul"
usn = "1MS24CS001"
age = 20
marks = 85.5

print("Student Name:", name)
print("USN:", usn)
print("Age:", age)
print("Marks:", marks)
print("Data Type:", type(marks))
```

## 2. Control Structures

Control structures determine the execution flow of a program.

### 2.1 Conditional statements

- `if`: Executes statements when a condition is true.

- `if-else`: Selects between two alternatives.
- `if-elif-else`: Selects from multiple alternatives.
- Nested if: Places one conditional statement inside another.

Example:

```
marks = 78

if marks >= 90:
    grade = "A+"
elif marks >= 80:
    grade = "A"
elif marks >= 70:
    grade = "B"
elif marks >= 60:
    grade = "C"
else:
    grade = "F"

print("Grade:", grade)
```

## 2.2 Looping statements

| Loop | Purpose |
|---|---|
| `for` | Iterates over a sequence |
| `while` | Repeats while a condition is true |
| `break` | Terminates the loop |
| `continue` | Skips the current iteration |
| `pass` | Placeholder statement |

Example:

```
marks = [78, 85, 92, 45, 67]

for mark in marks:
    if mark >= 70:
        print(mark, " - Qualified")
    else:
        print(mark, " - Not Qualified")
```

### 3. Functions in Python

A function is a reusable block of code designed to perform a specific task.

Advantages:

- Code reusability.

- Modularity.
- Reduced code duplication.
- Easier testing and debugging.

## Types of functions

1. Built-in functions: `print()`, `len()`, `sum()`, `max()`.
2. User-defined functions.
3. Functions with arguments.
4. Functions with return values.
5. Lambda functions.

Example:

```
def calculate_average(marks):
    return sum(marks) / len(marks)

student_marks = [80, 75, 90, 85]

average = calculate_average(student_marks)

print("Average Marks:", average)
```

### 4. Strings in Python

A string is an immutable sequence of characters enclosed in single, double or triple quotation marks.

#### Important string operations

| Operation | Example | Result |
|---|---|---|
| Length | `len("Python")` | 6 |
| Uppercase | `"python".upper()` | PYTHON |
| Lowercase | `"PYTHON".lower()` | python |
| Strip spaces | `" Ravi ".strip()` | Ravi |
| Replace | `"CS".replace("C","IT")` | ITS |
| Split | `"A,B,C".split(",")` | `['A','B','C']` |
| Find | `"Python".find("t")` | 2 |
| Slicing | `"Python"[0:3]` | Pyt |

#### String manipulation example

```
student = "  Rahul Kumar  "

name = student.strip()

print("Original:", student)
print("Processed:", name)
```

```
print("Uppercase:", name.upper())
print("Lowercase:", name.lower())
print("Length:", len(name))
print("First Name:", name.split()[0])
```

### 5. Lists in Python

A list is an ordered, mutable collection of elements. Lists can contain different data types and allow duplicate values.

**Important list operations**

| Operation | Description |
|---|---|
| `append()` | Adds an element |
| `insert()` | Inserts an element at a position |
| `extend()` | Adds multiple elements |
| `remove()` | Removes a specified element |
| `pop()` | Removes an element by index |
| `sort()` | Sorts elements |
| `reverse()` | Reverses the list |
| `len()` | Returns list length |
| `sum()` | Calculates total |
| `max()` / `min()` | Finds maximum/minimum |

Example:

```
marks = [75, 88, 65, 92, 80]

marks.append(95)
marks.remove(65)
marks.sort()

print("Updated Marks:", marks)
print("Total:", sum(marks))
print("Highest:", max(marks))
print("Lowest:", min(marks))
```

### 6. List Comprehensions

List comprehension provides a concise way to create a new list from an existing iterable using a single expression.

General syntax:

```
new_list = [expression for item in iterable if condition]
```

## Examples

**Example 1: Generate squares**

```
numbers = [1, 2, 3, 4, 5]

squares = [x*x for x in numbers]

print(squares)
```

**Example 2: Filter even numbers**

```
numbers = [10, 15, 20, 25, 30, 35]

even_numbers = [x for x in numbers if x % 2 == 0]

print(even_numbers)
```

**Example 3: Categorize students**

```
marks = [85, 45, 76, 32, 90, 55]

passed = [m for m in marks if m >= 40]
failed = [m for m in marks if m < 40]

print("Passed:", passed)
print("Failed:", failed)
```

**Lab Program 1: Student Record Creation and Processing**

Aim: Write a Python program to create synthetic student records and display student details using variables, data types, input and output operations.

Program:

```
students = [
    ["Rahul", 85, 20],
    ["Arun", 45, 21],
    ["Kiran", 72, 19],
    ["Manoj", 32, 20],
    ["Priya", 92, 21]
]

print("STUDENT RECORDS")
print("---------------------------")

for s in students:
    name, marks, age = s

    print("Name:", name)
    print("Marks:", marks)
    print("Age:", age)
    print("---------------------------")
```

Expected output:

```
STUDENT RECORDS
---------------------------
Name: Rahul
Marks: 85
Age: 20
---------------------------
Name: Arun
Marks: 45
Age: 21
...
```

**Concepts covered: Variables, data types, lists, nested lists and iteration.**

**Lab Program 2: Categorization Using Control Structures**

Aim: Develop a Python program to categorize student records based on marks using conditional statements.

Program:

```
students = [
```

```
    ["Rahul", 85],
    ["Arun", 45],
    ["Kiran", 72],
    ["Manoj", 32],
    ["Priya", 92]
]

for name, marks in students:

    if marks >= 90:
        category = "Excellent"
    elif marks >= 75:
        category = "Very Good"
    elif marks >= 60:
        category = "Good"
    elif marks >= 40:
        category = "Pass"
    else:
        category = "Fail"

    print(name, marks, category)
```

**Concepts covered: `if-elif-else`, nested lists, loops and data categorization.**

**Lab Program 3: Functions for Student Data Analysis**

Aim: Write user-defined functions to calculate average marks, identify the highest scorer and categorize students.

Program:

```
students = [
    ["Rahul", 85],
    ["Arun", 45],
    ["Kiran", 72],
    ["Manoj", 32],
    ["Priya", 92]
]

def calculate_average(records):
    marks = [s[1] for s in records]
    return sum(marks) / len(marks)

def find_topper(records):
    return max(records, key=lambda x: x[1])
```

```
def categorize(records):
    for name, marks in records:
        if marks >= 40:
            print(name, "Pass")
        else:
            print(name, "Fail")

print("Average:", calculate_average(students))
print("Topper:", find_topper(students))
print("Student Categories:")
categorize(students)
```

**Concepts covered: User-defined functions, arguments, return values, `max()`, lambda expressions and list processing.**

### Lab Program 4: String Manipulation and Record Validation

Aim: Develop a Python program to process synthetic employee records using string operations and validate employee information.

Program:

```
employees = [
    "EMP101,Rahul Kumar,Developer",
    "EMP102,ARUN SHARMA,Tester",
    "EMP103, Kiran Rao ,Analyst",
    "EMP104,Manoj,Manager"
]

for record in employees:

    fields = record.split(",")

    emp_id = fields[0].strip()
    name = fields[1].strip().title()
    designation = fields[2].strip().upper()

    if emp_id.startswith("EMP") and name and designation:
        print(emp_id, name, designation)
    else:
        print("Invalid Record")
```

Expected output:

```
EMP101 Rahul Kumar DEVELOPER
EMP102 Arun Sharma TESTER
```

```
EMP103 Kiran Rao ANALYST
EMP104 Manoj MANAGER
```

**Concepts covered: String splitting, stripping, capitalization, uppercase conversion, validation and conditional statements.**

**Lab Program 5: List Operations for Data Processing**

Aim: Write a Python program to process synthetic network traffic records and identify high-traffic and low-traffic observations.

Consider the following synthetic network traffic data representing packet counts.

Program:

```
traffic = [120, 450, 80, 900, 350, 1100, 250, 750]

high_traffic = []
low_traffic = []

for packets in traffic:

    if packets >= 500:
        high_traffic.append(packets)
    else:
        low_traffic.append(packets)

print("Traffic Records:", traffic)
print("High Traffic:", high_traffic)
print("Low Traffic:", low_traffic)

print("Total Packets:", sum(traffic))
print("Maximum Traffic:", max(traffic))
print("Minimum Traffic:", min(traffic))
print("Average Traffic:", sum(traffic)/len(traffic))
```

**Concepts covered: List creation, append, iteration, conditional processing, sum, max and min.**

**Lab Program 6: Data Filtering Using List Comprehensions**

Aim: Develop a Python program to filter, categorize and transform synthetic network traffic records using list comprehensions.

Program:

```
traffic = [120, 450, 80, 900, 350, 1100, 250, 750]

# Filter high traffic records
high = [x for x in traffic if x >= 500]

# Filter low traffic records
low = [x for x in traffic if x < 500]

# Convert packet counts into percentage values
percentage = [(x / sum(traffic)) * 100
              for x in traffic]

# Categorize traffic
category = [
    "High" if x >= 500 else "Normal"
    for x in traffic
]

print("Original Traffic:", traffic)
print("High Traffic:", high)
print("Low Traffic:", low)
print("Percentage:", percentage)
print("Categories:", category)
```

**Concepts covered: List comprehensions, conditional expressions, filtering, transformation and categorization.**

8. Integrated Laboratory Exercise – Synthetic Data Record Management

Aim: Design and implement a Python-based synthetic student data management system to validate, categorize, filter and analyse student records using Python basics, control structures, functions, strings, lists and list comprehensions.

program:

```
records = [
    ["ST101", "Rahul", 85, "CSE"],
    ["ST102", "Arun", 35, "ISE"],
    ["ST103", "Kiran", 72, "CSE"],
    ["ST104", "Priya", 95, "ECE"],
    ["ST105", "Manoj", 55, "ISE"],
    ["ST106", "Anita", 28, "ECE"]
]

# Function to categorize students
def categorize(marks):
```

```
    if marks >= 85:
        return "Excellent"
    elif marks >= 60:
        return "Good"
    elif marks >= 40:
        return "Pass"
    else:
        return "Fail"

# Process records
print("STUDENT DATA ANALYSIS")
print("--------------------------------")

for record in records:
    sid = record[0].strip().upper()
    name = record[1].strip().title()
    marks = record[2]
    dept = record[3].strip().upper()

    category = categorize(marks)

    print(sid, name, marks, dept, category)

# Filter passed students
passed = [r for r in records if r[2] >= 40]

# Filter failed students
failed = [r for r in records if r[2] < 40]

# Find CSE students
cse_students = [r for r in records if r[3] == "CSE"]

# Find students scoring above 75
top_students = [r[1] for r in records if r[2] > 75]

# Calculate average marks
average = sum(r[2] for r in records) / len(records)

print("\nPassed Students:", len(passed))
print("Failed Students:", len(failed))
print("CSE Students:", cse_students)
print("Students scoring above 75:", top_students)
print("Average Marks:", average)
```